Einführung: Algorithmen, Übersetzung und das erste Programm

PROG · KAPITEL 01 · SPICKZETTEL

ALGORITHMUS

Eine genaue, endliche Schrittfolge, die ein Problem löst. Jeder Schritt ist ohne Rückfrage ausführbar.

Anforderungen: **Endlichkeit**, **Ausführbarkeit**, **Korrektheit**, **Effizienz**.

Begriff	Bedeutung
terminierend	endet nach endlich vielen Schritten
determiniert	gleiche Eingabe → gleiches **Ergebnis**
deterministisch	nächster Schritt immer eindeutig → gleicher **Weg**

Merksatz: Deterministisch ist immer auch determiniert, umgekehrt nicht.

DATENSTRUKTUREN

Struktur	Idee	Beispiel
Array	gleicher Typ, hintereinander, Index	Matrizen
Liste	Elemente verweisen aufeinander	dynamische Sammlungen
Baum	Knoten mit Unterknoten	Dateisystem
Graph	Knoten und Kanten	Routenplanung
Stack	LIFO: Letztes zuerst raus	Zurück-Taste
Queue	FIFO: Erstes zuerst raus	Druckaufträge, Rechnungen nach Eingang

In Python eingebaut: `list`, `tuple`, `dict`, `set`.

PARADIGMEN

Paradigma	Idee	Beispiel
imperativ/prozedural	Schritt für Schritt	C
objektorientiert	Objekte mit Daten und Methoden	Java
funktional	Funktionen ohne Seiteneffekte	Haskell
deklarativ/logisch	beschreiben, was erreicht werden soll	SQL

Python ist **multiparadigmatisch**.

VOM QUELLCODE ZUM PROGRAMM

Variante	Ablauf	Beispiele
Interpreter	Anweisung für Anweisung sofort ausführen	Shell, PHP
Compiler	vorab alles in Maschinencode	C, C++, Rust, Go
Zwischencode	Bytecode, eine VM führt ihn aus	Java, C#, Python

- Compiler: schnell, aber vor jedem Test kompilieren, nur eine Zielplattform.
- JIT: VM übersetzt oft genutzten Code zur Laufzeit in Maschinencode.
- Python (CPython): Quellcode → Bytecode → Python-VM.

WERKZEUGE

Werkzeug	Aufgabe	Aufruf
uv	Python, Projekte, Pakete, starten	`uv run hallo.py`
ruff	formatieren, Fehler finden	`uv run ruff check .`
ty	Datentypen prüfen	`uv run ty check`

Terminal: `pwd`, `ls` / `dir`, `cd ordner`, `cd ..`. Editor: Tab = 4 Leerzeichen, UTF-8.

DAS ERSTE PROGRAMM

```
# Kommentar
print("Hello World!")  # Hello World!
```

- Einrückung bildet Blöcke (4 Leerzeichen), Doppelpunkt leitet sie ein.
- Kein Semikolon, eine Zeile = eine Anweisung.
- Groß-/Kleinschreibung zählt: `Print` ≠ `print`.
- Konsole (`>>>`) zeigt Werte auch ohne `print`, Text mit Anführungszeichen: `'Hallo'`. In einer `.py`-Datei brauchst du `print()`.

SYNTAXFEHLER ODER LAUFZEITFEHLER?

Beim Start prüft CPython zuerst die **Syntax der ganzen Datei**, dann läuft sie Zeile für Zeile.

```
print("A")
Print("B")   # NameError in Zeile 2
print("C")
# Ausgabe: A, dann Fehler
```

```
print("A")
print("B"    # SyntaxError
print("C")
# Ausgabe: nur Fehler, kein "A"
```

FEHLERMELDUNG LESEN

1. Letzte Zeile: **was**? (z. B. `NameError`)
2. Zeile darüber: **wo**? (Datei, Zeile)
3. Hinweis lesen: `Did you mean: 'print'?`

TYPISCHE FEHLER

- Determiniert (Ergebnis) und deterministisch (Weg) verwechselt.
- LIFO und FIFO vertauscht.
- „Python ist eine reine Interpretersprache“: falsch, Python nutzt Zwischencode.
- Nicht ausführbarer Schritt wie „Rechnung prüfen“, kein Fall für Abweichungen.
- `Print` statt `print`, fehlendes Anführungszeichen oder Klammer.
- In der `.py`-Datei Ausdruck ohne `print()` und Ausgabe erwartet.
- Tabs und Leerzeichen gemischt.

GRUNDFORM

```
for i in range(3):
    print("Runde", i)  # läuft 3-mal
print("fertig")        # läuft 1-mal danach
# Runde 0 / Runde 1 / Runde 2 / fertig
```

- Eingerückt = läuft in **jeder** Runde.
- Nicht eingerückt = läuft **einmal nach** der Schleife.
- Doppelpunkt am Ende der `for`-Zeile.

RANGE-VARIANTEN

`range(start, stop, schritt)`: Start ist dabei, Stopp **nicht**.

Aufruf	Zahlen	Runden
`range(3)`	0, 1, 2	3
`range(4)`	0, 1, 2, 3	4
`range(1, 4)`	1, 2, 3	3
`range(1, 6)`	1, 2, 3, 4, 5	5
`range(2, 7, 2)`	2, 4, 6	3
`range(0, 10, 3)`	0, 3, 6, 9	4
`range(2, 9, 3)`	2, 5, 8	3

Merksatz: Für „1 bis n“ schreibst du `range(1, n + 1)`.

LISTE DURCHGEHEN

```
einkauf = ["Kaffee", "Tee", "Kakao"]
for ware in einkauf:
    print("Ich kaufe", ware)
# Ich kaufe Kaffee / ... Tee / ... Kakao
```

Runde	ware	Ausgabe
1	"Kaffee"	Ich kaufe Kaffee
2	"Tee"	Ich kaufe Tee
3	"Kakao"	Ich kaufe Kakao

So viele Runden, wie die Liste Elemente hat, auch wenn du die Variable nicht benutzt.

MUSTER AUFSUMMIEREN

1. **Vor** der Schleife: Startwert (`summe = 0`, Produkt: `1`).
2. **In** der Schleife: dazuzählen.
3. **Nach** der Schleife: `print` oder `return`.

```
preise = [3, 5, 2]
summe = 0
for preis in preise:
    summe = summe + preis
print("Gesamt:", summe)  # Gesamt: 10
```

Runde	preis	summe vorher	summe nachher
Start	–	–	0
1	3	0	3
2	5	3	8
3	2	8	10

`summe += preis` ist dasselbe in kurz.

MUSTER ZÄHLEN

```
zahlen = [4, 7, 1, 9]
anzahl = 0
for zahl in zahlen:
    if zahl > 5:
        anzahl += 1  # nur im if
print(anzahl)  # 2
```

Runde	zahl	zahl > 5?	anzahl
Start	–	–	0
1	4	nein	0
2	7	ja	1
3	1	nein	1
4	9	ja	2

RUNDENTABELLE ANLEGEN

1. Spalte für Runde, je eine Spalte pro Variable, die sich ändert.
2. Zeile „Start“: Werte vor der Schleife.
3. Pro Runde Zeile für Zeile durchgehen, neue Werte eintragen.
4. Eigene Spalte für die Ausgabe.
5. Am Ende: Was passiert nach der Schleife?

Merksatz: Der Wert am Ende einer Runde ist der Wert am Anfang der nächsten.

TYPISCHE FEHLER

- Off-by-one: `range(1, 5)` liefert nur 1 bis 4.
- Vergessen, dass `range` bei 0 beginnt.
- `summe = 0` **in** der Schleife: Am Ende steht nur der letzte Wert (bei `[3, 5, 2]` → 2).
- `return summe` in der Schleife: nach Runde 1 Schluss.
- Produkt mit Startwert 0: Ergebnis immer 0.
- `anzahl += 1` nicht tiefer als das `if` eingerückt: zählt jede Runde.
- Zeile nach der Schleife versehentlich eingerückt: läuft in jeder Runde.

Variablen, Datentypen und Operatoren

DATENTYPEN

Typ	Beispiele	Wichtig
`int`	42, -7, 1_000_000	beliebig groß
`float`	3.14, 2.0	binär gespeichert, ungenau
`str`	"Hallo", ""	Text, unveränderlich
`bool`	True, False	großgeschrieben
`None`	None	kein Wert, nicht 0, nicht ""

```
print(type(3.14))       # <class 'float'>
print(0.1 + 0.2 == 0.3) # False
print(ord("A"), chr(66))  # 65 B
```

Postleitzahlen, Telefonnummern: `str`. Python ist dynamisch typisiert, mit Typangaben + ty graduell.

VARIABLEN

`=` heißt „zeigt ab jetzt auf“. Rechte Seite wird zuerst ausgerechnet.

```
zahl = 10
zahl = zahl + 1    # 11
a, b = 1, 2
a, b = b, a        # a = 2, b = 1
wert = 3,5         # Tupel (3, 5)!
alter: int = 25    # Typangabe, prüft ty
```

- Namen: Buchstaben, Ziffern, `_`, nicht mit Ziffer beginnen, kein Schlüsselwort.
- `snake_case` (Variablen, Funktionen), `PascalCase` (Klassen), `UPPER_SNAKE_CASE` (Konstanten).
- Konstante: `MWST_SATZ: Final = 0.19` (`from typing import Final`), ty meldet Neuzuweisung.
- Kommentar `#`, Docstring `"""…"""` unter `def` / `class`.

OPERATOREN

Ausdruck	Ergebnis	Regel
`7 / 2`	3.5	immer `float`
`6 / 3`	2.0	auch hier `float`
`7 // 2`	3	ganzzahlig, rundet **ab**
`-7 // 2`	-4	ab = zur kleineren Zahl
`7 % 3`	1	Rest
`2 ** 10`	1024	Potenz
`2 + 3 * 4`	14	Punkt vor Strich

- Gerade: `n % 2 == 0`.
- `x += 1` statt `x = x + 1`; kein `x++`.
- Vergleiche `== != < > <= >=` liefern `bool`. `is` nur für `None`.
- `18 <= alter <= 65` ist erlaubt.

Rangfolge: Klammern → `**` → `* / // %` (von links) → `+ -`.

```
print(2 + 17 % 5 * 2 // 3)  # 3
# 17%5=2, 2*2=4, 4//3=1, 2+1=3
```

STRINGS UND F-STRINGS

```
print("ha" * 3, len("Hallo"))  # hahaha 5
print("Hallo"[0], "Hallo"[-1]) # H o
print("Hallo"[1:3])            # al
print("ll" in "Hallo")         # True
```

Index ab 0, `[1:3]` ohne Position 3.

```
preis = 3.5
print(f"{preis:.2f} €")  # 3.50 €
```

TYPUMWANDLUNG

Ausdruck	Ergebnis
`int(9.99)`	9 (schneidet ab)
`int("42")`	42
`float(5)`	5.0
`str(42)`	'42'
`round(9.5)`	10
`5 + 2.0`	7.0
`True + 1`	2
`"12" + "3"`	'123'

Geld: `Decimal("19.99")` mit Anführungszeichen (`from decimal import Decimal`).

```
print(int(19.99 * 100))   # 1998
print(round(19.99 * 100)) # 1999
```

FEHLERARTEN

Art	Beispiel	Wann?
Syntax	`print("Hallo"`	vor dem Start
Typ	`"Alter: " + 36`	ty vorher, sonst `TypeError`
Laufzeit	`a / 0`	`ZeroDivisionError`
Logik	`5 - 3` statt `+`	nie automatisch

Traceback von unten lesen: letzte Zeile = was, darüber = wo.

TYPISCHE FEHLER

- `-7 // 2` für −3 gehalten (richtig −4).
- `n / 2 == 0` statt `n % 2 == 0`.
- `alter == "30"` ist `False`: Zahl ≠ Text.
- `"Alter: " + 36` gibt `TypeError`, besser f-String.
- `int()` rundet nicht, es schneidet ab.
- `true` statt `True`.
- Variable nur im übersprungenen `if` gesetzt: `NameError`.
- `5 + 2 / 2` statt `(5 + 2) / 2`.

IF, ELIF, ELSE

```
t = 22
if t < 0:
    print("Gefriert!")
elif t < 20:
    print("Kühl.")
elif t < 30:
    print("Angenehm.")  # läuft
else:
    print("Heiß!")
```

Merksatz: Python stoppt beim **ersten** wahren Zweig, also strengste Bedingung nach oben.

Testen an den Grenzen: 90/89, 75/74 usw.

MATCH UND BEDINGTER AUSDRUCK

```
note = "B"
match note:
    case "A" | "B" | "C":
        print("Bestanden")  # läuft
    case _:                 # wie default
        print("Ungültig")
```

Kein `break` nötig. Nur für feste Werte, Bereiche mit `if`.

```
betrag = 25.0
x = "offen" if betrag > 0 else "bezahlt"
# x = "offen"
```

Falsy: `False`, `None`, `0`, `0.0`, `""`, `[]`. Alles andere ist truthy, auch `"0"` und `[0]`.

LOGIK

- Rangfolge: Vergleiche → `not` → `and` → `or`.
- Umkehren: jede Teilbedingung umkehren, `and` ↔ `or`. Gegenteil von `>= 50` ist `< 50`.
- Kurzschluss: Absicherung nach vorn.

```
anzahl = 0
print(anzahl > 0 and 10 / anzahl > 1)
# False, keine Division
```

WHILE

Wiederholt, **solange** die Bedingung wahr ist. Für unbekannte Rundenzahl.

```
i = 1
while i <= 5:
    print(i, end=" ")
    i = i + 2
# 1 3 5
```

i vorher	i <= 5?	Ausgabe	i danach
1	ja	1	3
3	ja	3	5
5	ja	5	7
7	nein	Ende	7

Kein `do – while`: `while True:` mit `break`.

FOR, BREAK, CONTINUE

```
ergebnis = 1
for i in range(1, 5):  # 1, 2, 3, 4
    ergebnis = ergebnis * i
print(ergebnis)  # 24
```

```
for i in range(1, 6):
    if i == 3:
        continue   # nur diese Runde
    if i == 5:
        break      # ganze Schleife
    print(i)
# 1 / 2 / 4
```

LISTEN

Ausdruck	`n = [15, 13, 8, 10, 17]`
`n[0]` / `n[-1]`	15 / 17
`n[1:3]`	[13, 8]
`len(n)`	5
`sum(n)`, `min(n)`, `max(n)`	63, 8, 17
`8 in n`	True
`sorted(n)`	neue sortierte Liste
`n.append(12)`	hängt hinten an
`n[5]`	`IndexError`

Gültig: Index 0 bis `len(n) - 1`.

```
tage = ["Mo", "Di"]
werte = [18.5, 21.0]
for i, tag in enumerate(tage):
    print(i, tag)       # 0 Mo / 1 Di
for t, w in zip(tage, werte):
    print(t, w)         # Mo 18.5 / Di 21.0
```

Tupel `(3, 4)`: lesen wie Liste, nicht änderbar.

MAXIMUM SELBST FINDEN

```
werte = [-3, -1, -7]
hoechster = werte[0]  # nicht 0!
for wert in werte:
    if wert > hoechster:
        hoechster = wert
print(hoechster)  # -1
```

TYPISCHE FEHLER

- Doppelpunkt oder Einrückung vergessen.
- `>= 50` vor `>= 90` geprüft: alle bekommen „ausreichend“.
- `while`-Variable nie geändert: Endlosschleife (Strg+C).
- `break` und `continue` verwechselt.
- Division vor der Absicherung: `ZeroDivisionError`.
- Maximum mit 0 gestartet: falsch bei negativen Werten.
- `[[0] * 4] * 3` kopiert nur den Verweis: Ändern einer Zeile ändert alle.
- `"0"` für falsy gehalten.

Funktionen und Rekursion

DEFINIEREN UND AUFRUFEN

```
def addiere(a: int, b: int) -> int:
    ergebnis = a + b
    return ergebnis

print(addiere(5, 7))  # 12
```

Teil	Bedeutung
`def`	Definition beginnt
`a`, `b`	Parameter (Platzhalter)
`5`, `7`	Argumente (Werte beim Aufruf)
`-> int`	Rückgabetyp, prüft nur ty
`return`	beendet, liefert Ergebnis

Merksatz: `def` legt nur an, erst `name(...)` führt aus.

```
def main() -> None:
    print("Start")

if __name__ == "__main__":
    main()  # nur bei direktem Start
```

PRINT ODER RETURN?

	`print(x)`	`return x`
zeigt etwas an?	ja	nein
weiterrechnen möglich?	nein (`None`)	ja
beendet Funktion?	nein	ja, sofort

```
def f() -> None:
    print(119.0)

a = f()     # zeigt 119.0
print(a)    # None
```

- Ohne `return` (auch in einem Zweig): `None`.
- Mehrere Werte = Tupel: `return min(z), max(z)`, dann `k, g = ...`.
- `break` verlässt die Schleife, `return` die ganze Funktion.

GÜLTIGKEIT UND PARAMETER

- Variablen in der Funktion sind **lokal** (eigener Rahmen auf dem Aufrufstapel).
- `if` / `for` bilden keinen eigenen Bereich, Funktionen schon. Suche: LEGB.
- Globale Variable in der Funktion zuweisen: `UnboundLocalError`.
- Parameter zeigt auf **dasselbe Objekt** wie das Argument.

```
def aendere(liste: list[int]) -> None:
    liste[0] = 99      # außen sichtbar
    liste = [7, 7, 7]  # nur lokal

z = [1, 2, 3]
aendere(z)
print(z)  # [99, 2, 3]
```

STANDARDWERTE STATT ÜBERLADUNG

Keine Überladung: Zweites `def` mit gleichem Namen ersetzt das erste.

```
def brutto(netto, satz=0.19):
    return round(netto * (1 + satz), 2)

print(brutto(100))            # 119.0
print(brutto(100, 0.07))      # 107.0
print(brutto(satz=0.07, netto=100))  # 107.0
```

```
def summe(*zahlen: int) -> int:
    ergebnis = 0
    for zahl in zahlen:
        ergebnis += zahl
    return ergebnis

print(summe(1, 2, 3, 4))  # 10
```

Standardwert-Parameter stehen hinten. Liste als Standardwert: stattdessen `None` und in der Funktion `[]`.

IMPERATIV UND APPLIKATIV

	imperativ	applikativ
Frage	Wie?	Was ist das Ergebnis?
Zustand	ändert sich	bleibt
Wiederholung	`for`, `while`	Rekursion

REKURSION

Pflicht: **Abbruchbedingung** und **Aufruf mit kleinerem Problem**.

```
def fakultaet(n: int) -> int:
    if n == 0:        # Basisfall
        return 1
    return n * fakultaet(n - 1)

print(fakultaet(3))  # 6
```

Aufruf	wartet auf	liefert
fakultaet(3)	fakultaet(2)	$3 \cdot 2 = 6$
fakultaet(2)	fakultaet(1)	$2 \cdot 1 = 2$
fakultaet(1)	fakultaet(0)	$1 \cdot 1 = 1$
fakultaet(0)	–	1

Tabelle von unten nach oben lesen. Ohne Abbruch: `RecursionError` (ca. 1000 Rahmen).

TESTEN

```
assert fakultaet(5) == 120  # nichts
assert fakultaet(3) == 7    # AssertionError
```

pytest: `uv add --dev pytest`, Datei `test_….py`, Funktionen `test_…`, `uv run pytest`. Randfälle mittesten (leere Liste, Exponent 0).

TYPISCHE FEHLER

- `print` statt `return`: Ergebnis ist `None`.
- `main` ohne Klammern: nichts passiert.
- Fehlendes `return` in einem Zweig.
- Zwei `def addiere`: `TypeError` beim Aufruf, kein `SyntaxError`.
- Liste als Standardwert: wird zwischen Aufrufen geteilt.
- Basisfall `return 0` bei der Fakultät: alles wird 0.
- `fakultaet(-1)`: läuft an `n == 0` vorbei, `RecursionError`.
- Nur ein Test: kann zufällig bestehen.

Klassen und Objekte (OOP I)

BEGRIFFE

Begriff	Bedeutung
Klasse	Bauplan, `class Konto:` (PascalCase)
Objekt / Instanz	konkretes Exemplar, `Konto("A", 100)`
Attribut	Daten, Zustand: `a.stand`
Methode	Funktion in der Klasse: `a.einzahlen(50)`
`__init__`	Konstruktor, legt Attribute an
`self`	das Objekt links vom Punkt

Merksatz: Jedes Objekt hat eigene Attributwerte, die Methoden stehen nur einmal in der Klasse.

KLASSE MIT METHODEN

```
class Konto:
    def __init__(self, inhaber, stand=0):
        self.inhaber = inhaber
        self.stand = stand

    def einzahlen(self, betrag):
        self.stand += betrag

a = Konto("A", 100)
b = Konto("B")
a.einzahlen(50)
print(a.stand, b.stand)  # 150 0
```

- Erzeugen ohne `new`. `a.einzahlen(50)` = `Konto.einzahlen(a, 50)`.
- Genau ein `__init__`, Varianten über Standardwerte.

AUSGABE: STR UND REPR

Methode	für	benutzt von
`__str__`	Menschen	`print()`, f-Strings
`__repr__`	Entwickler	Konsole, Listen

`__str__` gibt mit `return` einen Text zurück. Ohne `__str__` zeigt `print(a)` nur `<__main__.Konto object at 0x…>`.

NAMEN UND KLASSENATTRIBUTE

`k2 = k1` erzeugt kein neues Konto, nur einen zweiten Namen: Änderung über `k2` ist bei `k1` sichtbar, `k1 is k2` ist `True`. Bei eigenen Klassen vergleicht `==` zunächst nur die Identität.

```
class Konto:
    anzahl = 0          # Klassenattribut

    def __init__(self):
        Konto.anzahl += 1  # Klassenname

Konto()
Konto()
print(Konto.anzahl)  # 2
```

KAPSELUNG MIT PROPERTIES

- `_name`: intern (nur Abmachung), `__name`: Name Mangling. Kein echtes `private`.
- Property: Methode, die wie ein Attribut benutzt wird (ohne Klammern).

```
class Konto:
    def __init__(self, stand):
        self._stand = 0
        self.stand = stand  # nutzt Setter

    @property
    def stand(self):         # Getter
        return self._stand

    @stand.setter
    def stand(self, wert):   # Setter
        if wert < 0:
            print("Ungültig")
            return
        self._stand = wert

k = Konto(100)
k.stand = -5      # Ungültig
print(k.stand)    # 100
```

Situation	Lösung
frei änderbar	öffentliches Attribut
nur lesbar	`_attr` + Property ohne Setter
Änderung mit Regeln	`_attr` + Property mit Setter
berechnet	Property ohne eigenes Attribut

HAT-EIN, MODULE, @DATACLASS

- **Hat ein**: Attribut mit anderem Objekt (`self.kunde = kunde`), Verweis, keine Kopie.
- Modul = `.py`-Datei, Paket = Ordner mit `__init__.py`: `from banking.konto import Konto`.

```
from dataclasses import dataclass

@dataclass
class Kunde:
    name: str
    ort: str

k = Kunde("A", "Nord")
print(k)  # Kunde(name='A', ort='Nord')
print(k == Kunde("A", "Nord"))  # True
```

`@dataclass` erzeugt `__init__`, `__repr__`, `__eq__`. Für Datenbehälter; für Verhalten mit Regeln normale Klasse mit Properties.

TYPISCHE FEHLER

- `self` in der Parameterliste vergessen: `TypeError`.
- `self.` vor dem Attribut vergessen.
- In `__str__` `print` statt `return`.
- `self.anzahl += 1` legt ein Instanzattribut an, Klassenzähler bleibt.
- Tippfehler `a.stnad = 500`: still neues Attribut.
- Property mit Klammern aufgerufen: `TypeError`.
- Im Setter `self.stand = …` statt `self._stand`: `RecursionError`.
- `@dataclass` prüft keine Werte.

Vererbung und Polymorphie (OOP II)

VERERBUNG: „IST EIN“

Unterklasse übernimmt Attribute und Methoden der Oberklasse und ergänzt Neues. Sparkonto **ist ein** Konto → Vererbung. Konto **hat einen** Kunden → Attribut.

```
class Konto:
    def __init__(self, inhaber, stand=0):
        self.inhaber = inhaber
        self.stand = stand

    def einzahlen(self, betrag):
        self.stand += betrag

    def abheben(self, betrag):
        if betrag > self.stand:
            return False
        self.stand -= betrag
        return True

class Sparkonto(Konto):   # erbt
    def __init__(self, inh, stand, zins):
        super().__init__(inh, stand)
        self.zins = zins   # neu

s = Sparkonto("A", 1000, 2)
s.einzahlen(500)          # geerbt
print(s.stand, s.zins)    # 1500 2
```

Merksatz: Eigenes `__init__` → `super().__init__(…)` ist Pflicht, Python ruft es nicht selbst auf.

`isinstance(s, Konto)` → True (zählt Unterklassen mit), `type(s) is Konto` → False (nur genaue Klasse).

ÜBERSCHREIBEN

Gleicher Name, gleiche Parameter, in der Unterklasse.

```
from typing import override

class Girokonto(Konto):
    def __init__(self, inh, stand, dispo):
        super().__init__(inh, stand)
        self.dispo = dispo

    @override
    def abheben(self, betrag):
        if betrag > self.stand + self.dispo:
            return False
        self.stand -= betrag
        return True

g = Girokonto("B", 100, 500)
print(g.abheben(400))  # True, stand -300
print(g.abheben(300))  # False
```

- Suche: Klasse des Objekts → Oberklasse → … → `object`. Erste Fundstelle gewinnt (`__mro__`).
- `super().methode()` benutzt die Version der Oberklasse mit.
- `@override`: Python ignoriert es, ty findet damit Tippfehler.

Merksatz: **Überladen** (gleicher Name, andere Parameter, dieselbe Klasse) gibt es in Python nicht, **Überschreiben** schon.

POLYMORPHIE

Derselbe Aufruf, verschiedenes Verhalten je nach Klasse des Objekts.

```
konten = [Konto("A", 100),
          Girokonto("B", 100, 500)]
for k in konten:
    print(k.abheben(400))
# False (Konto) / True (Girokonto)
```

Duck Typing: Python prüft beim Ausführen nur, ob die Methode existiert, nicht die Klasse.

OBJECT UND ABSTRAKTE KLASSEN

Jede Klasse erbt von `object` (Standard-`__str__`, `__repr__`, `__eq__`).

```
from abc import ABC, abstractmethod

class Zahlungsart(ABC):
    @abstractmethod
    def bezahlen(self) -> str: ...

Zahlungsart()  # TypeError
```

- Keine Objekte einer abstrakten Klasse, Fehler schon **beim Erzeugen**.
- Jede Unterklasse muss alle `@abstractmethod` überschreiben, sonst bleibt sie abstrakt.

FÄHIGKEITEN STATT INTERFACES

Python hat kein `interface`, erlaubt aber mehrere Oberklassen: erst Kategorie, dann Fähigkeiten.

```
class Druckbar(ABC):
    @abstractmethod
    def drucken(self) -> None: ...

class Dokument: ...

class Lieferschein(Dokument, Druckbar):
    @override
    def drucken(self) -> None:
        print("druckt")

d = Lieferschein()
if isinstance(d, Druckbar):
    d.drucken()  # druckt
```

- **Kategorie** („ist ein“): konkrete und abstrakte Methoden, Attribute, Konstruktor, eine je Klasse.
- **Fähigkeit** („kann“): nur abstrakte Methoden, beliebig viele je Klasse.

TYPISCHE FEHLER

- `super().__init__` vergessen: `AttributeError` erst bei der ersten Methode.
- `super.__init__(…)` ohne Klammern hinter `super`.
- Unterklasse überschreibt Attribut der Oberklasse (`self.stand = laufzeit`).
- Import von `override`, `ABC`, `abstractmethod` vergessen (`NameError`) oder `(ABC)` vergessen.
- Fähigkeit ohne `isinstance` genutzt: `AttributeError`.

AUFBAU STUFE 1

Keine Vererbung, nur „hat ein“.

Klasse	hat	Methoden
Lager	viele Produkte	`anzeigen`, `finden`, `suchen`
Warenkorb	Kunde, Positionen	`hinzufuegen`, `entfernen`, `gesamtsumme`, `ist_leer`, `anzeigen`
Position	Produkt, Menge	`summe`

Startpaket: `produkt.py` und `daten.py` nur benutzen, `main.py` ausbauen (Menü mit `while True` und `match`).

MUSTER FINDEN

```
def finden(nummern, gesucht):
    for n in nummern:
        if n == gesucht:
            return n    # sofort raus
    return None         # nach der Schleife!

liste = [101, 102, 203, 204]
print(finden(liste, 203))  # 203
print(finden(liste, 999))  # None
```

Runde	n	== 203?	was passiert
1	101	nein	weiter
2	102	nein	weiter
3	203	ja	`return`, Ende

Merksatz: „Nicht gefunden“ darfst du erst sagen, wenn du alle angeschaut hast.

MUSTER SAMMELN

```
def suchen(namen, text):
    treffer = []
    for name in namen:
        if text.lower() in name.lower():
            treffer.append(name)
    return treffer

namen = ["Kaffee 1 kg", "Grüner Tee",
         "Kaffeemühle"]
print(suchen(namen, "kaffee"))
# ['Kaffee 1 kg', 'Kaffeemühle']
print(suchen(namen, "Zucker"))  # []
```

Leere Liste vor, `append` in, `return` nach der Schleife. Kein Treffer = `[]`, nicht `None`.

HINZUFUEGEN() SCHRITT FÜR SCHRITT

1. `schon_im_korb` = Menge, die schon im Korb liegt.
2. `schon_im_korb + menge > bestand`? Dann `return False`, nichts ändern.
3. Noch keine Position: neue `Position` anhängen. Sonst `position.menge += menge`. Dann `return True`.

Bestand 3:

Aufruf	im Korb	Prüfung	Ergebnis
menge 5	0	0 + 5 > 3	False
menge 3	0	0 + 3 > 3 nein	True
menge 1	3	3 + 1 > 3	False

`hinzufuegen()` liefert nur `True` / `False`, die Meldung schreibt `main.py`.

MUSTER AUFSUMMIEREN

```
summe = 0
for p in [74.61, 46.38]:
    summe += p
print(round(summe, 2))  # 120.99
```

Im Projekt: `summe = Decimal(0)`, dann `summe += position.summe()`.

MENÜ UND BESTELLEN

- Lager, Kunde, Warenkorb **vor** der `while`-Schleife anlegen.
- Bestellen: leerer Korb → Meldung, `return`. Sonst für jede Position `verkaufen(menge)`, Summe ausgeben, Korb leeren.
- Frühes `return` beendet nur den Menüpunkt.

```
eingabe = "abc"
print(eingabe.isdigit())  # False
print("999".isdigit())    # True
```

STUFE 2: POLYMORPHIE

`Produkt` wird abstrakt mit `mwst_satz()`. `Lebensmittel`: 7 %, `Zubehoer`: 19 %. `bruttopreis` steht nur einmal in `Produkt` und ruft `self.mwst_satz()`.

```
from decimal import Decimal
p = Decimal("12.99") * Decimal("1.07")
print(p.quantize(Decimal("0.01")))  # 13.90
```

NAMEN ZEIGEN AUF OBJEKTE

```
korb = ["2 x Kaffee"]
bestellung = korb      # keine Kopie
korb.clear()
print(bestellung)      # []
korb = ["1 x Tee"]
best2 = korb
korb = []              # neue Liste
print(best2)           # ['1 x Tee']
```

Merksatz: `clear()` leert das Objekt für alle Namen, `= []` bindet nur einen Namen neu.

TYPISCHE FEHLER

- `else: return None` direkt unter dem `if`: Suche endet nach dem ersten Produkt.
- `return None` in der Schleife statt danach.
- Nur `menge > bestand` geprüft, Menge im Korb vergessen.
- Immer `append`: dasselbe Produkt als zwei Positionen.
- `korb = Warenkorb(kunde)` in der Schleife: Korb jedes Mal leer.
- `korb.positionen.clear()` leert auch die Bestellung, richtig `korb.positionen = []`.
- Kein Treffer als `None` statt `[]` zurückgegeben.